

Mini panel de Contact Center

Construye un servicio que centralice la actividad de un equipo de atención y la convierta en métricas confiables. Queremos ver cómo piensas la arquitectura, no cuánto código produces.

ROL	NÚCLEO EVALUADO	STACK
FULL-STACK (NODE)	ARQUITECTURA Y DISEÑO	TÚ LO ELIGES
IA		TIEMPO
ANTIGRAVITY: HTTPS://ANTIGRAVITY.GOOGLE/		1H - 2H

CONTEXTO

En WeKall ayudamos a las empresas a centralizar voz, chat e IA en una sola plataforma. Una pieza de ese trabajo es **Engage 360**, nuestro contact center: convierte cada interacción de un equipo de atención en datos accionables.

Para esta prueba vas a construir una versión mínima de esa idea: un servicio que registre la actividad de atención (llamadas y tickets) y exponga métricas operativas. El reto no está en la cantidad de funciones, sino en **modelar bien el dominio** y en que las métricas sean correctas cuando los datos crecen.

Puedes usar IA libremente. No la limitamos. Lo que evaluamos es tu criterio: qué decisiones tomas, cuáles descartas y por qué. La IA acelera el código; el diseño y las decisiones siguen siendo tuyos.

EL PROBLEMA

Un equipo de soporte atiende a clientes a través de llamadas y tickets. Cada agente trabaja una jornada y resuelve interacciones a lo largo del día. La operación es en Colombia (zona horaria `UTC-5`). El líder del equipo necesita un panel para responder tres preguntas:

- ¿Cuántas interacciones atendió cada agente y cuántas quedaron resueltas?
- ¿Cuál es el tiempo promedio de resolución por agente?
- ¿Cómo se comporta el volumen de interacciones **por día** en un rango de fechas?

Tu trabajo es diseñar y construir el backend que modela estos datos y los expone vía API, más una interfaz mínima que los muestre.

REQUISITOS FUNCIONALES

MUST-HAVE

Gestión de interacciones

- Crear una interacción (tipo `llamada` o `ticket`), asignada a un agente, con timestamp de apertura.
- Cambiar su estado: `abierta` → `en_progreso` → `resuelta`. Al resolverse se registra el timestamp de cierre.
- Listar interacciones con **filtros** (agente, estado, rango de fechas) y **paginación**.

NÚCLEO – AQUÍ MIRAMOS DE CERCA

Endpoint de métricas

Un endpoint que reciba un rango de fechas y devuelva, por agente: total de interacciones, total resueltas, tasa de resolución y **tiempo promedio de resolución** (diferencia entre cierre y apertura).

Además, una serie de **volumen por día** dentro del rango.

Dos cosas que separan una solución sólida de una frágil: (1) que la agregación siga siendo correcta y eficiente con muchos registros — no traer todo a memoria y sumar en un `for`; (2) que el agrupamiento "por día" respete la zona horaria de la operación (`UTC-5`), no la del servidor. Una interacción de las 8 p.m. en Cali pertenece a ese día, no al siguiente.

MUST-HAVE

Interfaz mínima

Una vista que liste interacciones con sus filtros y una vista de métricas (tabla o gráfico simple). No buscamos diseño pulido; buscamos que el frontend consuma tu API con sensatez y maneje los estados de carga y error.

PAUTAS TÉCNICAS

- **Backend en Node.** El framework, el lenguaje sobre Node (JS o TS), la base de datos y el ORM/driver son **tu decisión**. Justifícala en el documento de decisiones.
- **Frontend a tu elección** (React, Vue, o lo que prefieras). Puede ser sencillo.
- Diseña el **modelo de datos** pensando en cómo se consultará, no solo en cómo se guarda.
- Incluye un script o *seed* que cargue datos de ejemplo suficientes para que las métricas se vean (cientos de interacciones, varios agentes, fechas que crucen medianoche).
- Maneja errores y entradas inválidas de forma explícita. No asumas el camino feliz.

LIBERTAD Y LÍMITES

Eres libre de: elegir todo el stack, usar IA, usar librerías, estructurar el proyecto como creas correcto.

No evaluamos: que uses una tecnología específica, ni el pulido visual, ni que cubras funciones extra. Preferimos un alcance acotado y bien resuelto antes que mucho a medias.

ENTREGABLES

1. **El software** — repositorio Git con el código, instrucciones para levantarlo y el seed de datos. Que arranque con pasos claros.

2. **Documento de decisiones** (`DECISIONS.md`) — el flujo de diseño y construcción. Es tan importante como el código (ver abajo).
3. **README** — cómo instalar, levantar y probar; endpoints disponibles.

Qué esperamos en el documento de decisiones

Cuéntanos el **razonamiento**, no solo el resultado. Responde con honestidad:

- **Arquitectura general:** cómo organizaste el proyecto y por qué (capas, responsabilidades, dónde vive la lógica de negocio).
- **Modelo de datos:** qué entidades definiste, qué relaciones, y por qué ese modelo facilita las consultas de métricas.
- **El endpoint de métricas:** cómo resolviste la agregación y la zona horaria. Qué alternativas consideraste y descartaste.
- **Trade-offs:** decisiones donde elegiste una opción sabiendo que tenía costos.
- **Uso de IA:** en qué te apoyaste, qué te entregó que estuvo mal o incompleto, y qué corregiste o validaste tú.
- **Qué harías distinto** con más tiempo o en producción.

CÓMO LO EVALUAMOS

CRITERIO	QUÉ MIRAMOS	PESO
Arquitectura	Separación de responsabilidades, claridad estructural, dónde vive cada cosa	25
Calidad de código	Legibilidad, nombres, manejo de errores, consistencia	20
Núcleo: métricas	Agregación correcta y eficiente, zona horaria bien resuelta, casos borde	20
API y modelo de datos	Diseño de endpoints, modelado del dominio, filtros y paginación	15

CRITERIO	QUÉ MIRAMOS	PESO
Documento de decisiones	Claridad del razonamiento, trade-offs honestos, uso reflexivo de IA	15
Tests y DX	Pruebas donde aportan, README claro, facilidad para levantarlo	5

Un proyecto pequeño, bien pensado y bien explicado supera a uno grande y desordenado. La claridad de lo simple.

ENTREGA

Comparte el repositorio (link a GitHub/GitLab o un `.zip`) con el `DECISIONS.md` incluido. Si algo del enunciado te parece ambiguo, decide con criterio y déjalo anotado: esa decisión también es parte de lo que evaluamos.